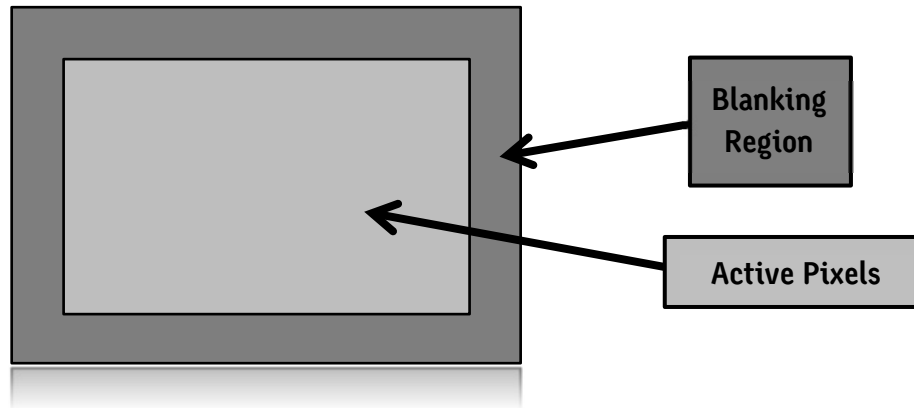


A Boring Intro Line:

In order to output a certain resolution to a monitor we will need the **X** and **Y** number of pixels of this mode, the **Refresh Rate** (Hz) and also quite a few other values that relate to the **syncing** part of the process.

Blanking Basics:

So, a region around the image you want to output has to be blank.



The surrounding **blanking region** can take up as much area as **25%** of the **Video Frame**. This is essential for the display to be able to recognize, center and output the actual **Active Region** which is really the contents of the resolution we are outputting.

Surrounding Area Contents:

Let's now dive into the contents of this **Blanking Region**.

Going from left to right (Horizontally), we get the **HSync** that lasts a few pixels, then, the **Horizontal Back Porch** that lasts a few pixels as well, right before the **Active Pixels** whose amount equals to the **Horizontal Resolution** of the format, followed by the **Horizontal Front Porch** at the right end of the line.

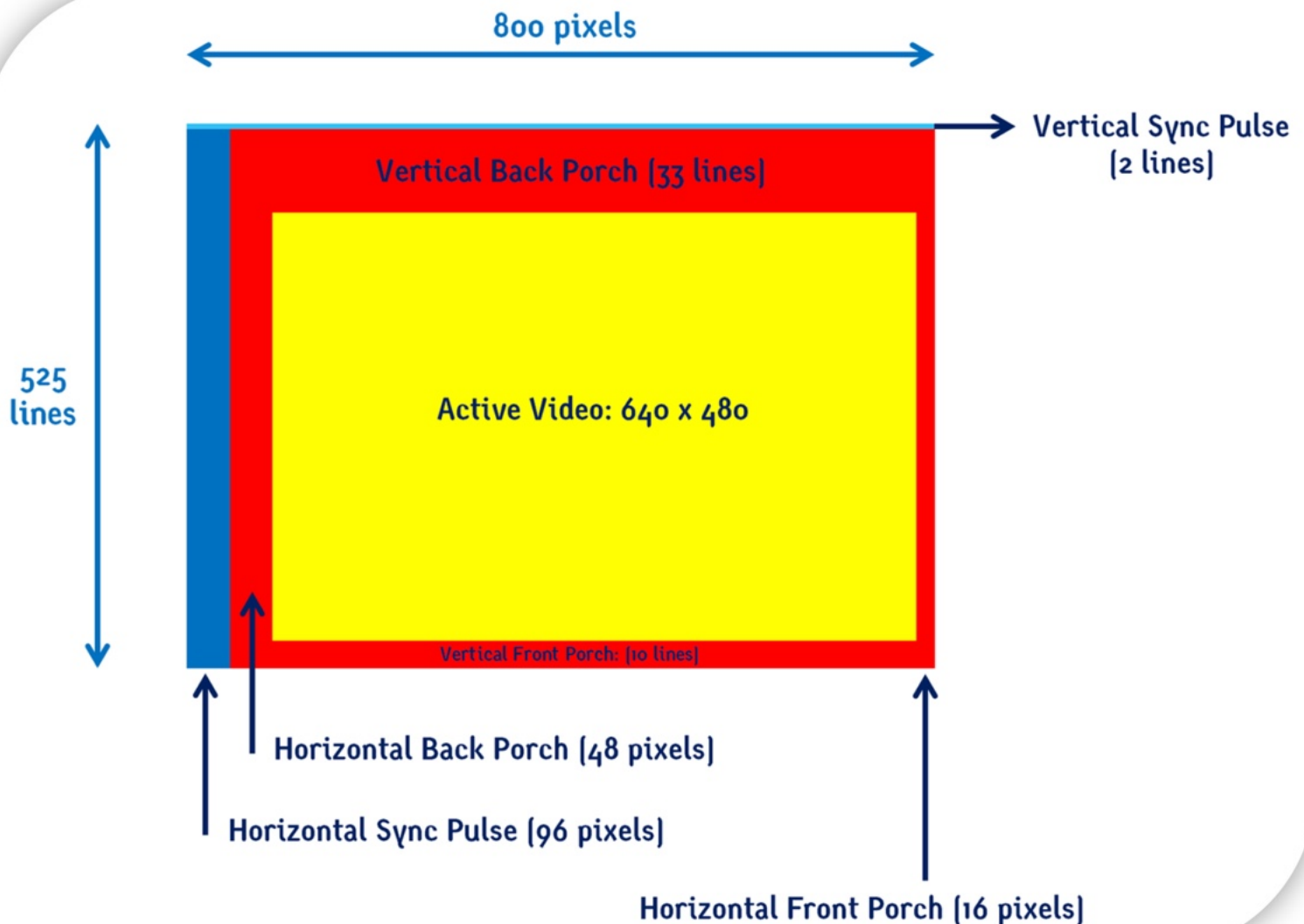
And, from top to bottom, we get the **VSync**, that lasts for a couple lines, then the **Vertical Back Porch**, and then a number of lines equaling the **Vertical Resolution** of the format followed by the **Vertical Front Porch**.

The **H,V Front** and **Back porches** are the blank area we earlier talked about and the **HSync** & **VSync** are **signals** that tell the monitor to start drawing the next **Horizontal Row** or **Vertical Line**.

So the GPU drives the Display by Generating an **H_Sync** signal, the **H_Back_Porch** area, the **H_Active_Pixels** and the **H_Front_Porch** area, repeatedly for every line.

And the number of **total lines** = **VSync** lines + **V_Back_Porch** lines + **V_Active_Pixels** lines + **V_Front_Porch** lines.

(This will make sense later when examining the structure of **.vfs** files).



In this example, in order to display a 640x480 image, we need $(640+96+48+16) \times (480+2+33+10) = 800 \times 525$ total (pixels)x(lines).

Reduced Blanking:

Oh well, CRT Monitors did require a “Full Fat” Blanking area so that the Electrons’ Gun had enough μ seconds to stop firing while it changed line and also enough time to prepare to start drawing the next sequence of active pixels on the following line.

But these days, it is a fact that Flat Panel Displays, even though they work fine with such Blanking settings, can Sync and output the desired Active Resolution with much lower Blanking bezels.

Q: Why bother reducing the Blanking area, since it is optional?

A: Cause with Reduced Blanking, we have a lower total number of (pixels)x(lines) generated by the computer to display the same Active Resolution. This leads to lower Pixel Clock speeds and puts less stress on the Video Adapter. Also, in some cases, it results to crisper image, as huge Blanking bezels can lead to noisy video output.

Modelines:

A **Modeline** is a neat, compact way of fitting all this information in a **single line** of text. We will cover this subject via an example.

Let's say we have this **Modeline**:

"1600x1024@60.00" 108.0 1600 1616 1656 1704 1024 1027 1030 1056 -Hsync -Vsync
Resolution@Hz PixelClock H1 H2 H3 H4 V1 V2 V3 V4 low low

This is a **Modeline** for the 1600x1024 Resolution @ 60Hz.

From this format, we can extract the following quantities:

TotalPixelsPerLine = H4 (1704)

HSync = H3-H2 (40)

Horizontal Back Porch = H4-H3 (48)

Horizontal Front Porch = H2-H1 (16)

ActivePixelsPerLine = H1 (1600)

TotalLinesPerFrame = V4 (1056)

VSynC = V3-V2 (3)

Vertical Back Porch = V4-V3 (26)

Vertical Front Porch = V2-V1 (3)

ActualLinesPerFrame = V1 (1024)

The **PixelClock** speed (MHz) is automatically calculated by the compiler, and depending on the hardware, there is a limit on how high this value can be.

In practice, this **limits** the **highest resolution** possible the **Video Adapter** may **drive the display** at.

That is cause its value increases as the Hx and Vx values get higher and, of course, has an upper limit.

Obviously, we can generate a **Modeline** if we know the opposite parameters.

So the **Modeline** for the 640x480 Resolution of the image on the previous page would be:

"640x480@60.00" (PixelClock) 640 656 752 800 480 490 492 525 -Hsync +Vsync

This information is very useful in when generating and compiling custom resolutions for a monitor.

In order to obtain the Modelines for the different resolution formats supported by a monitor, one can refer to the manufacturer's **Timing Manual** or **Service Manual**.

A second way is to use a tool like "SwitchResX" for OSX or "PowerStrip" for Windows to read the **EDID information** directly from the Display (supposing DDC is functioning well).

There is not just a single valid Modeline for each **Resolution@RefreshRate** mode of a monitor, cause we can manually adjust the Blanking to reduce or even increase it.

That results to many possible (similar though) combinations.

(More on that later).

General Rules & Basic Transformations:

Horizontal Rules:

- a) $\text{TotalPixelsPerLine} = \text{ActivePixelsPerLine} + \text{HSync} + \text{Horizontal Back Porch} + \text{Horizontal Front Porch}$
- b) $H1 < H2 < H3 < H4$
- c) All of these above numbers can be perfectly divided by 8.

For example, you cannot have a HSync value of 92.

That is cause $92/8 = 11.5$. Note that the Horizontal value of all used Active Resolutions obeys this rule. For example in 640x480, 1920x1080, 1680x1050, 1600x1024, 1280x1024 etc, the Horizontal number of pixels can be perfectly divided by 8.

This rule does not apply for the Vertical dimensions (V1, V2, V3, V4).

Left/Right Shifting:

In order to slightly move the image to the left, we need to add increments of 8 to the Horizontal Front Porch and subtract equal increments of 8 to the Horizontal Back Porch. (The actual TotalPixelsPerLine will remain the same).

To move to the right, add increments of 8 to the Horizontal Back Porch and subtract equal pixels from the Horizontal Front Porch.

This, however is not really useful, cause most modern Displays Auto-Center the image via an Auto-Adjust Button.

So, one can skip the manual L-R shifting thing.

Vertical Rule:

$$V1 < V2 < V3 < V4$$

Up/Down Shifting:

Shall the picture be too high, add a small value to the Vertical Back Porch number of lines and subtract the same value from the Vertical Front Porch number.

If it appears too low, do the opposite.

Again, this can be skipped, cause the Auto-Adjust will center the image vertically, as well.

Out-of-Proportions Fix:

In case the image appears shrunk (narrow), decrease small increments of 8 from the TotalPixelsPerLine, without changing any other Hx value.

Some compilers (ie VFC) will complain because then the first Horizontal rule becomes broken, but a video format will be created none-the-less.

If it appears wider than it should, then proceed by adding small increments of 8 to the TotalPixelsPerLine.

This did not work for me, but it depends on the Graphics Chip and Display combination, I guess.

VFC Intro:

VFC is a command-line based **Video Format Compiler** tool available on **Irix**. It lets you **compile** and create a **custom Resolution & Timing Format** to drive your display, based on **source text files**.

Installation & General Information:

To **install VFC**, run the **Software Manager**, navigate to the **VFC_6.5.22** or **VFC_6.5.30** folder (depending on the OS' version) and go to the **Customize Installation** tab to select it. (It is not included in the **Standard Installation** by default).

Ok! So, the vfc executable will be located in the **/usr/sbin** folder.

Also, there is a vfc folder in the **/usr/gfx/ucode** System Folder.

This one contains a folder named **rules** and another folder named **vfs**.

The rules folder contains the **.def** rules files for various **Chip & Board types**.

And the **vfs** folder contains **.vfs** text **templates** (source files).

The idea is to **create a .vfs** file with all the correct values for every parameter and then **compile** it according to the manufacturer's rules for our system's **Video Chip** and **Video Board**.

When done, a **binary .vfo** file is created on the **/** folder and needs to be placed here:

/usr/gfx/ucode/xxx/vof

The **xxx** folder depends on the system configuration: ie xxx = ODSY for all Odyssey VPros, CRM for O2s, etc..

Then we can hopefully see it in the **Display Properties** system panel and apply it.

In case a **.vfo** format located in the **../vof** folder does not appear on the Display Properties' **Valid Formats** list, it simply is incompatible for some reason.

3-Ways to approach compiling a .vfo file:

The Lazy way - By using the **BlockSync.vfs** template:

So, let's say we need to **compile** a custom **.vfo** for the **Odyssey V12** at **1680x1050@60Hz**.

We can use the **BlockSync.vfs** template without modifying it just by **passing parameters** to the compiler while executing this line on a Console:

```
/usr/sbin/vfc -p "-DACTIVE_LINES=1050" -p "-DACTIVE_PIXELS=1680" -p "-DPFS=60" -c  
chip=VPro_Chip.def,board=VPro_Board.def -o 1680x1050_60.vfo  
/usr/gfx/ucode/vfc/vfs/BlockSync.vfs
```

Then, the 1680x1050_60.vfo file will be generated on the **root folder**.

This method is **fast** and does not include expert knowledge of the stuff we talked about in the previous pages, but on the downside there is **no control over the Blanking**, etc..

The Intermediate way - Use a custom .vfs file, but get all the values we need directly from our Monitor's EDID information or a Modeline Calculator:

We will base our .vfs on an existing example by SGI. So, let's duplicate the 1280x1024_60.vfs (inside the /usr/gfx/ucode/vfc/vfs folder) and rename it to 1680x1050.vfs

That's the structure of this .vfs:

```
/*
** 1280x1024_60.vfs - SGI standard format
*/
```

Commented-Out

General

```
{
    FieldsPerFrame = 1;
    FramesPerSecond = 60;
    TotalLinesPerFrame = 1065;
    TotalPixelsPerLine = 1680;
    ActiveLinesPerFrame = 1024;
    ActivePixelsPerLine = 1280;
    FormatName = "1280x1024_60";
}
```

1 Field - non-Interlaced Formats

FPS = 60Hz

All other values pretty in line with what we've already covered.

Active Line {

```
    HorizontalFrontPorch = 40 pixels;
    HorizontalSync = 120 pixels;
    HorizontalBackPorch = 240 pixels;
}
```

TotalPixelsPerLine = ActivePixelsPerLine +
HorizontalFrontPorch + HorizontalSync +
HorizontalBackPorch

Field

```
{
    Vertical Sync =
    {
        {
            Length = 1.0H;
            Low = 0.0 usec;
        }
        repeat 2
    {
        Length = 1.0H;
    }
    }

    Initial Low;
    Vertical Back Porch =
    {
        {
            Length = 1.0H;
            High = HorizontalSync;
        }
        repeat 34
    {
        Length = 1.0H;
        Low = 0.0 usec;
        High = HorizontalSync;
    }
    }
}
```

Vertical Sync = 2

(The VSync value is the number just after "repeat" on this section)

Vertical Back Porch = 36

It is a VFC requirement that the V_Back_Porch gets written decreased by 2, compared to what it actually is.

So: TotalLinesPerFrame = VSync +
(VerticalBackPorch + 2) + VerticalFront Porch

```

Initial High;

Active =
{
repeat 1024
{
    Length = 1.0H;
    Low = 0.0 usec;
    High = HorizontalSync;
}
}

Vertical Front Porch =
{
    repeat 3
    {
        Length = 1.0H;
        Low = 0.0 usec;
        High = HorizontalSync;
    }
}

#if 0
    postprocess
    {
        dump edge;
    }
#endif

```

This Value is the ActiveLinesPerFrame (1024)

Vertical Front Porch = 3

In the following example, we will compile a **custom .vfo** for the **Odyssey V12** at **1680x1050@60Hz**.

The first step would be to **obtain the values** we need.

One way is to **read the EDID information** from the **Display** using **Windows** or **OSX** (via **PowerStrip** or **SwitchResX**).

This requires **DDC communication** between the **monitor** and the **Video Card** of a **PC** or a **Mac**.

Connect the monitor you wish to use with your SGI Computer to -say- your mac, go to the **SwitchResX Control Panel** and dump the monitor's **EDID** to a text file. (All values show there).

Or connect it to a **Windows PC** and use **PowerStrip**. Go to **options** **Monitor information...** and select the drop-down option: **Read data directly form monitor**.

On Windows, one can also use **NVidia's Control Panel** **WorkStation** **Topology** tab to find this **EDID information**.

The "**ideal**" **values** may, as well, be extracted directly from the **Monitor's Timing** or **Service Manual**, but these are hard to get sometimes.

But, for now, we are going to use a **Modeline Calculator** to generate values that should work with almost all Monitors that support the **Resolution** and **Refresh Rate** we wish to **output**.

There's a good online tool here: <https://arachnoid.com/modelines/>

Enable "**Computation details**", enter the desired **Resolution** & **Refresh Rate** values and click on "**Compute**":



The Calculator

For those who are in a hurry and who know what a "modeline" is, here's the calculator. The documentation is below.

Required inputs:

Horizontal resolution pixels	1680
Vertical resolution pixels	1050
Refresh rate Hz	60

Options:

FB output mode (default is xf86 mode)	☐
Interlaced mode	☐
Image margin (to correct overscan)	☐
Computation details	☒

Results:

[compute](#)

```
# 1680x1050 @ 60.00 Hz (GTF) hsync: 65.22 kHz; pclk: 147.14 MHz
Modeline "1680x1050_60.00" 147.14 1680 1784 1968 2256 1050 1051 1054 1087 -HSync +Vsync
```

Details:

```
10: [V FIELD RATE] : 60.000000
14: [V FRAME RATE] : 60.000000
15: [LEFT MARGIN (PIXELS)] : 0.000000
16: [RIGHT MARGIN (PIXELS)] : 0.000000
17: [TOTAL ACTIVE PIXELS] : 1680.000000
18: [IDEAL DUTY CYCLE] : 25.400184
19: [H BLANK (PIXELS)] : 576.000000
20: [TOTAL PIXELS] : 2256.000000
21: [PIXEL FREQ] : 147.136320
22: [H FREQ] : 65.220000
17: [H SYNC (PIXELS)] : 184.000000
18: [H FRONT PORCH (PIXELS)] : 104.000000
36: [V ODD FRONT PORCH(LINES)] : 1.000000
```

So:

TotalLinesPerFrame = 1087;
 TotalPixelsPerLine = 2256;
 ActiveLinesPerFrame = 1050;
 ActivePixelsPerLine = 1680;
 HorizontalFrontPorch = 104 pixels;
 HorizontalSync = 184 pixels;
 HorizontalBackPorch = 288 pixels;
 Vertical Sync = 3
 Vertical Back Porch = 33 - 2 = 31
 Vertical Front Porch = 1

And the contents of the resulting .vfs are:

```
General
{
    FieldsPerFrame = 1;
    FramesPerSecond = 60;
    TotalLinesPerFrame = 1087;
    TotalPixelsPerLine = 2256;
    ActiveLinesPerFrame = 1050;
    ActivePixelsPerLine = 1680;
    FormatName = "1680x1050_60";
}

Active Line {
    HorizontalFrontPorch = 104 pixels;
    HorizontalSync = 184 pixels;
    HorizontalBackPorch = 288 pixels;
}

Field
{
    Vertical Sync =
    {
        {
            Length = 1.0H;
            Low = 0.0 usec;
        }
        repeat 3
    {
        Length = 1.0H;
    }
    }

    Initial Low;
    Vertical Back Porch =
    {
        {
            Length = 1.0H;
            High = HorizontalSync;
        }
        repeat 31
    {
        Length = 1.0H;
        Low = 0.0 usec;
        High = HorizontalSync;
    }
    }

    Initial High;

    Active =
    {
        repeat 1050
    {
        Length = 1.0H;
        Low = 0.0 usec;
        High = HorizontalSync;
    }
    }
}
```

```

    Vertical Front Porch = 1
    {
        repeat 1
        {
            Length = 1.0H;
            Low = 0.0 usec;
            High = HorizontalSync;
        }
    }
}

#if 0
    postprocess
    {
        dump edge;
    }
#endif

```

Now that the .vfs is ready, we may compile:

```

/usr/sbin/vfc -c chip=VPro_Chip.def,board=VPro_Board.def -o 1680x1050_60.vfo
/usr/gfx/ucode/vfc/vfs/1680x1050.vfs

```

Although this method produces **mathematically correct values** and therefore good .vfo files, it might actually not work, cause the Video Card of our SGI computer (espcesially older systems) could be subject to inadequate **Pixel Clock Speed** for certain HD Resolutions, or other **limitations**, as we are going to see shortly.

The Die-Hard way - Build a **custom .vfs** with **modified Blanking**:

This is the method to follow on the upcoming examples.

I do use the **Modeline Calculator** as a starting point and then modify the values to achieve the lowest Pixel Clock possible.

Various Chip & Board definitions depending on Hardware:

Onyx2 InfiniteReality:

```

/usr/sbin/vfc -c chip=voc1.def,board=dg4.def -o 1680x1050_60.vfo
/usr/gfx/ucode/vfc/vfs/1680x1050.vfs

```

VPro Odyssey (V6, V8, V10, V12) - **V6 & V8 Limit: PxlClk<109MHz or PxlClk >193MHz:**

```

/usr/sbin/vfc -c chip=VPro_Chip.def,board=VPro_Board.def -o 1680x1050_60.vfo
/usr/gfx/ucode/vfc/vfs/1680x1050.vfs

```

O2 - Limit: 2160 TotalPixelsPerLine:

```

/usr/sbin/vfc -c chip=O2_Chip.def,board=O2_Board.def -o 1680x1050_60.vfo
/usr/gfx/ucode/vfc/vfs/1680x1050.vfs

```

Octane or Indigo2 Mardi Gras (EMXI, ESSI, ESI ..) - Limit: PxlClk<160MHz:
/usr/sbin/vfc -c chip=mgras_vc3.def,board=mgras.def -o 1680x1050_60.vfo
/usr/gfx/ucode/vfc/vfs/1680x1050.vfs

A few Tips & Troubleshooting before we start:

There is a chance we might set as the “power-on” default format one that is not supported by our monitor.

The Refresh Rate may be sky high -or too low-, or there is something else that the monitor does not like and it just says: “Out of Range” while showing a Black Screen.

So everytime we reboot, after the OS is loaded, we just see this Black Screen and the annoying error message.

Let’s say this happened while trying to switch to our custom .vfo file: 1680x1050.vfo

The way around this is to “Stop for Maintenance” after we press the Power Button and go to the “Command Monitor”.

Enter “single” to boot to single user mode.

When asked, give the root password and there we are: The Single User Mode Command Line.

Type: “cd /usr/gfx/ucode/xxx/vof”

As mentioned earlier, the xxx depends on the system we are running: ie xxx = ODSY for all Odyssey VPros, CRM for O2s, etc..

Then enter: “cp 1280x1024_60.vfo 1680x1050.vfo”

This command will copy the failsafe 1280x1024.vfo over our bad 1680x1050.vfo

In case we are unsure about the exact names we may enter: “ls” to see all files that are located inside this vof folder, prior to executing the “cp” command.

Enter “reboot”, and the Computer will now restart using the failsafe 1280x1024_60.vfo instead.

As we play with the values on our .vfs text file (and save, then compile, then try the resulting .vfo file, repeat..) our display might think the input resolution is not what the computer actually outputs, but instead 1280x1024.

So, the (ActivePixelsPerLine)x(ActiveLinesPerFrame) might be 1680x1050, but the monitor may think it actually is 1280x1024 and show a stretched image.

This means that either we have gone too high or too low with the HorizontalFrontPorch, HorizontalSync and HorizontalBackPorch values, or that the VerticalBackPorch value is too low.

Our general goal, is to drop the HorizontalFrontPorch, HorizontalSync, HorizontalBackPorch and VerticalBackPorch values as low as possible, as long as the monitor actually recognizes the desired resolution and does not show a stretched, distorted 1280x1024 image, instead.

That way, we achieve a lower Pixel Clock setting (because the TotalPixesPerLine and TotalLinesPerFrame are relatively low) and get a crispier image without noise.

Not to mention that we put less stress on the adapter.

Always add or subtract increments of 8 to all Horizontal values.

This rule, as previously noted, does not apply to the Vertical ones.

In any case:

$\text{TotalPixelsPerLine} = \text{ActivePixelsPerLine} + \text{HorizontalFrontPorch} + \text{HorizontalSync} + \text{HorizontalBackPorch}$

So, when changing a value of the right part of the equation make sure you update the **TotalPixelsPerLine** value, as well

Same for this rule: $\text{TotalLinesPerFrame} = \text{VSync} + (\text{VerticalBackPorch} + 2) + \text{VerticalFront Porch}$

Again: the +2 is a VFC requirement.

So, on the **VerticalBackPorch** repeat loop, we have to insert the actual **VerticalBackPorch** value reduced by 2.

When compiling the .vfo file it might give you some **errors**.

If it says: "... **Check your Math**" it means that at least one of the two rules above does not checksum OK.

In this case, just redo the calculation and update the **TotalPixelsPerLine** or the **TotalLinesPerFrame** value.

Sometimes, it might complain that the Blanking margins are too low and tell you that it failed and "... **no Format was Created**".

Ignore it and check the **root (/)** folder: You will see that it has actually generated the .vfo.

Each time a .vfo file gets created on the **/** folder, we need to move it to the **/usr/gfx/ucode/xxx/vof** folder and **restart** the **Display Properties** panel to test it out.

Obviously, the syntax of the vfc command we are going to be using is this one:

/usr/sbin/vfc -c chip={our chip's .def file},board={our board's .def file} -o {name of the output .vfo file} {path to the .vfs source file}

In this example:

**/usr/sbin/vfc -c chip=VPro_Chip.def,board=VPro_Board.def -o 1680x1050_60.vfo
/usr/gfx/ucode/vfc/vfs/1680x1050.vfs**

our **Video Chip** is a **VPro**, and the **Video Board** is a **VPro Board** (**V6**, **V8**, **V10** or **V12**), the .vfo file will be named as: **1680x1050_60.vfo** and generated on the root folder and the .vfs **source file** is this one: **/usr/gfx/ucode/vfc/vfs/1680x1050.vfs**

It is a good idea to use the display's "**Auto-Adjust**" function each time we **load a new .vfo format**, cause the image may appear **off-center** or slightly **out of proportions** on our **Analog VGA monitor**. This is also advisable in case one uses a **converter box** to connect their SGI's **Analog Output** to a **Digital-only Display**.

It can be a slightly **tedious** process that involves **trial-and-error** and **experimenting**, but proves to be well worth it because of the **crisp image** output and **more screen real estate** we get when running our modern panel at its **native resolution**.

Mind you, even though the upcoming examples involve creating a valid .vfo format for a 1680x1050 and a 1600x1024 monitor, the thought process and the actual procedure are exactly the same for all possible resolutions, like 1920x1080, etc..

The claim is that should one understand the following examples, he could build his own, custom resolution within minutes.

The Frame Buffer Depth Setting:

Glancing at the **Display Properties** window, a section on the left controls the **Frame Buffer Depth**.

We may set this to **8 Bytes** or **16 Bytes**.

This setting, unlike Windows, does not have anything to do with the **Color Depth** (and so the number of colors we see on the screen) and is measured in **Bytes** and not in bits.

It is relevant to the size of several buffers, like **Z-Depth Buffer**, etc..

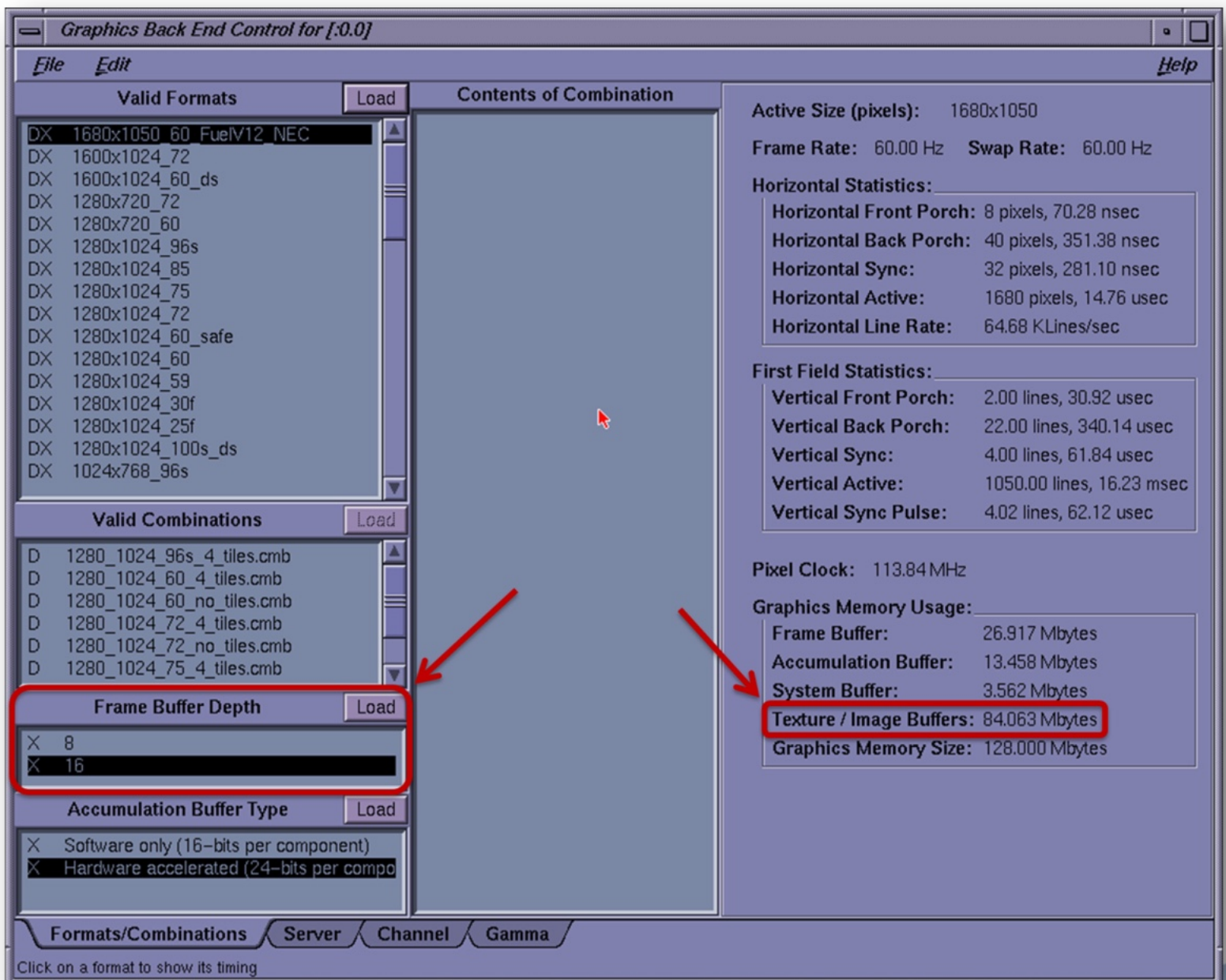
When doing **3D Modeling**, it is advised to keep it at **x16**.

If not, the **faces** of the **3D models** may appear distorted or see-through, etc..

However, keeping it at the high setting, takes up more space on the **Video Adapter's memory**.

And, although this is not a problem for a **V12**, a -say- **V10** that only has **32MB** of **Video memory**, cannot handle high resolutions at **x16** cause then no or very little Video memory is available for the **Texture / Image Buffers**.

Irix normally needs **more than 3MB** for these **Image Buffers**:



Shall this value be anything lower than that or even **negative**, we have to drop the **Frame Buffer Depth** setting down to **x8**.

For example, on a **V10**, for the **1680x1050** resolution, only **~1.5MB** is available for the **Texture & Image Buffers**, when the **Frame Buffer Depth** is set to **x16**.

The **Display Properties** panel will let us load this bad combination, but then Irix is going to freak out and not be able to properly **draw** all **elements** of the **Desktop** or the **Windows**.

So, depending on the **Video memory**, there are limitations on how high the Resolution can be cranked up when the **Frame Buffer Depth** is set to the **highest** setting.

And this only has to do with the **Active Horizontal & Vertical Resolution**, so **Reduced Blanking** or **Reduced Framerate** will have no effect here.

Real-World Examples:

Phew! Time to go down on some **examples**.

Reminder: All of the following stuff involves compiling **custom resolutions** just for **Single Monitor Setups**.

Fuel, V12, 1680x1050:

So, we need to create a **1680x1050** resolution for a **V12 Fuel**.

This **Video Board** is the one of the easiest possible to work on, since it really has no limitations regarding the **Video Memory** or the **Pixel Clock Speed** when it comes down to any resolution **up to 1920x1200**.

The starting point for the process is to go to the **/usr/gfx/ucode/vfc/vfs** folder and duplicate the **1280x1024_60.vfs** template, since it contains the entire code we need.

Rename the duplicate file to **1680x1050.vfs** and we may now edit this one.

Next, we will give ourselves a starting point by using a **Modeline Calculator**:

<https://arachnoid.com/modelines/index.html>

Input these values:

Required inputs:

Horizontal resolution pixels	1680
Vertical resolution pixels	1050
Refresh rate Hz	60

And by hitting **"compute"** we get the following **Modeline**:

Results:

```
# 1680x1050 @ 60.00 Hz (GTF) hsync: 65.22 kHz; pclk: 147.14 MHz
Modeline "1680x1050_60.00" 147.14 1680 1784 1968 2256 1050 1051 1054 1087 -HSync +Vsync
```

Cool, so the values we need are:

```
FramesPerSecond = 60;
TotalLinesPerFrame = 1087;
TotalPixelsPerLine = 2256;
ActiveLinesPerFrame = 1050;
ActivePixelsPerLine = 1680;
HorizontalFrontPorch = 1784-1680 = 104 pixels;
HorizontalSync = 1968-1784 = 184 pixels;
HorizontalBackPorch = 2256-1968 = 288 pixels;
Vertical Sync = 1054-1051 = 3
Vertical Back Porch = 1087-1054 -2 = 31
Active Vertical = ActiveLinesPerFrame = 1050
Vertical Front Porch = 1051-1050 = 1
```

Refer to the Modelines section in page 3 for details on how to extract these values from a Modeline.

Well, the **Display Properties** panel does not sometimes like formats that have a **TotalPixelsPerLine** value **greater than 2176**.

If we dump the values like this and **compile**, the resulting format might not even show on the **Valid Formats list**.

So, it is a good idea to reduce the **HorizontalBackPorch** value (that seems too high anyway) down to **208 pixels**, so that **TotalPixelsPerLine = 2176**.

Reminder: **TotalPixelsPerLine = ActivePixelsPerLine + HorizontalSync + HorizontalFrontPorch + HorizontalBackPorch**

We could now just drop these values in our **1680x1050.vfs** source file and save it:

```
/* My Custom 1680x1050 format! */

General
{
    FieldsPerFrame = 1;
    FramesPerSecond = 60;
    TotalLinesPerFrame = 1087;
    TotalPixelsPerLine = 2176;
    ActiveLinesPerFrame = 1050;
    ActivePixelsPerLine = 1680;
    FormatName = "1680x1050_60";
}

Active Line {
    HorizontalFrontPorch = 104 pixels;
    HorizontalSync = 184 pixels;
    HorizontalBackPorch = 208 pixels;
}

Field
{
    Vertical Sync =
    {
        {
            Length = 1.0H;
            Low = 0.0 usec;
        }
        repeat 3
        {
            Length = 1.0H;
        }
    }
}
```

```

    }

    Initial Low;
    Vertical Back Porch =
    {
        {
            Length = 1.0H;
            High = HorizontalSync;
        }
        repeat 31
        {
            Length = 1.0H;
            Low = 0.0 usec;
            High = HorizontalSync;
        }
    }

    Initial High;

    Active =
    {
        repeat 1050
        {
            Length = 1.0H;
            Low = 0.0 usec;
            High = HorizontalSync;
        }
    }

    Vertical Front Porch =
    {
        repeat 1
        {
            Length = 1.0H;
            Low = 0.0 usec;
            High = HorizontalSync;
        }
    }
}

#if 0
postprocess
{
    dump edge;
}
#endif

```

Then, on a **Console**, enter:

```
/usr/sbin/vfc -c chip=VPro_Chip.def,board=VPro_Board.def -o 1650x1050_60.vfo
/usr/gfx/ucode/vfc/vfs/1680x1050.vfs
```

Move the **1650x1050_60.vfo** to the **/usr/gfx/ucode/ODSY/vof** folder,

Launch the **Display Properties** and apply it.

This will work no problem cause a **V12** can handle the resulting **Pixel Clock** of **141.92MHz** just fine. However, because I am planning to use the **Fuel** with a modern Flat monitor, I could easily reduce the **Blanking area**, and as a result run at a **lower Pixel Clock** and achieve a slightly **crispier image**.

I need to lower the **HorizontalFrontPorch**, **HorizontalSync**, **HorizontalBackPorch** and **VerticalBackPorch** as much as possible.

So, will start reducing the Horizontal values little-by-little and save the .vfs and compile and apply it, and repeat.

Just a bit tedious, but straight-forward.

Always reduce these values by increments of 8, cause they should be perfectly divided by 8.

You will know when to stop cause, if you overdo it, the display will not be able to Sync at all or may show heavy streaking.

Another recommendation is to -generally- keep:

HorizontalFrontPorch < **HorizontalSync** < **HorizontalBackPorch**

This is not a mandatory thing to do, cause some manufacturers will give you Modelines straight out of their Timing Manuals that do not obey this rule, but for throughout my experimenting it proved to be just fine to follow.

In case you accidentally set a bad “power-on” default format refer to the previous “Tips & Troubleshooting” section to fix it.

A relatively safe starting point for LCDs is this one:

```
HorizontalFrontPorch = 32 pixels;
HorizontalSync = 48 pixels;
HorizontalBackPorch = 80 pixels;
```

Also update the

```
TotalPixelsPerLine = 1680 + 32 + 48 + 80 = 1840
```

Cool, now save the .vfs, compile, apply, and if everything works fine, try to even lower these values.

When compiling, it might complain about random stuff and even say: “No video format created”.. Nothing to worry about cause despite this warning it will create the .vfo on the root folder. See the “Tips & Troubleshooting” section to know what to alter should it give you “Verify your math” errors.

The Error factor gets printed on the Console output.

If this value is not 0, it will auto-adjust the Frame Rate to compensate.

Nothing to worry about here.

Each time you apply, run your monitor’s “Auto-Adjust” function to allow it to center the image, cause it might appear off-center or out-of-proportions when applied.

You know the Horizontal values combination is not good when after applying and “Auto-Adjusting”, on your monitor’s information (this can be accessed by the monitor’s menu) the resolution appears to be fine (in this example 1680x1050), but a part of the desktop is cropped or outside of the monitor boundaries.

This happened to me when I tried this combination:

```
HorizontalFrontPorch = 16 pixels;
HorizontalSync = 32 pixels;
HorizontalBackPorch = 48 pixels;
```

It really has to do with the exact monitor you are planning to use..

From now on, it is a matter of making subtle adjustments and experimenting.

So! For my display, the lowest possible values that work fine are:

```
HorizontalFrontPorch = 8 pixels;
HorizontalSync = 24 pixels;
HorizontalBackPorch = 40 pixels;
```

```
TotalPixelsPerLine = 1680 + 8 + 24 + 40 = 1752
```

That would be it for the optimal **Horizontal values**!

Now time to tinker with the **Vertical** ones.

This one is less complicated, but slightly tricky.

Right now, we are at this point:

```
Vertical Sync = 3
Vertical Back Porch = 31
Vertical Front Porch = 1
TotalLinesPerFrame = 3 + 31 + 2 + 1 = 1087
```

The goal is to keep the **Vertical Sync** and **Vertical Front Porch** values “as is”, and try to **reduce** the **Vertical Back Porch Value** (-1 at a time) until we get the lowest possible value that works with our monitor.

A value ain’t good when the display thinks that the resolution we are outputting is not the **1680x1050**, but **1280x1024**, instead.

To check whether this is the case, after applying the format from the **Display Properties panel**, go to your **monitor’s menu** (via the buttons on the device) and check the **information tab**.

Meanwhile, the image will appear a bit blurry and somewhat stretched or narrowed.

This is an indicator that the **Vertical Back Porch** value is bad.

To fix it just **subtract** or **add 1** to the **Vertical Front Porch** value (and -obviously- update the **TotalLinesPerFrame**, as well).

Starting from 31 and going down by 1 each time, I found out that the lowest possible value I could get my monitor to sync on **1680x1050** is 21.

In the meantime, there were other values in between 31 and 21 that would make it think it was a 1280x1024 resolution.

I just kept on subtracting 1 all the way down to 21.

So -at last- managed to write the format with the most reduced Blanking bezels possible, for this display.

Voila the final code of the .vfs source file:

```
/* Extremely Reduced Blanking - NEC E222W
** Fuel, V12
*/

General
{
    FieldsPerFrame = 1;
    FramesPerSecond = 60;
    TotalLinesPerFrame = 1077;
    TotalPixelsPerLine = 1752;
    ActiveLinesPerFrame = 1050;
    ActivePixelsPerLine = 1680;
    FormatName = "1680x1050_60";
}

Active Line {
    HorizontalFrontPorch = 8 pixels;
    HorizontalSync = 24 pixels;
    HorizontalBackPorch = 40 pixels;
}
```

```

Field
{
    Vertical Sync =
    {
        {
            Length = 1.0H;
            Low = 0.0 usec;
        }
        repeat 3
        {
            Length = 1.0H;
        }
    }

    Initial Low;
    Vertical Back Porch =
    {
        {
            Length = 1.0H;
            High = HorizontalSync;
        }
        repeat 21
        {
            Length = 1.0H;
            Low = 0.0 usec;
            High = HorizontalSync;
        }
    }

    Initial High;

    Active =
    {
        repeat 1050
        {
            Length = 1.0H;
            Low = 0.0 usec;
            High = HorizontalSync;
        }
    }

    Vertical Front Porch =
    {
        repeat 1
        {
            Length = 1.0H;
            Low = 0.0 usec;
            High = HorizontalSync;
        }
    }
}

#ifdef 0
    postprocess
    {
        dump edge;
    }
#endif

```

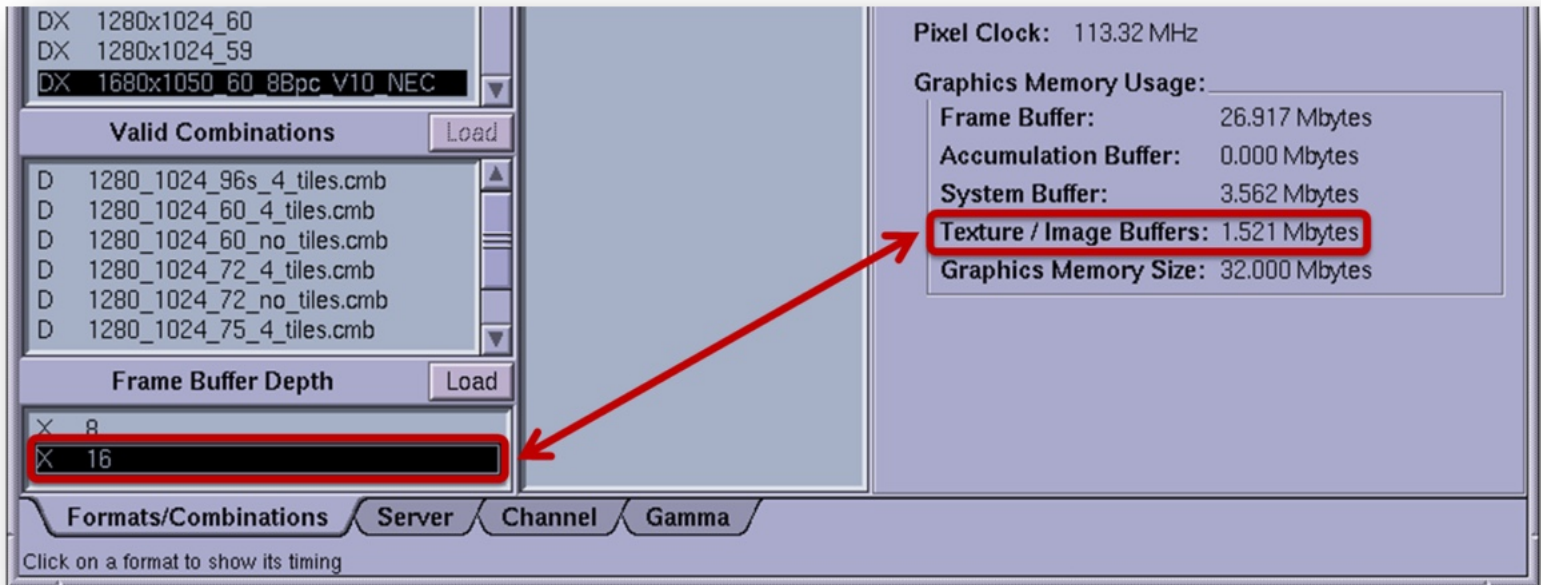
Octane2, V10, 1680x1050:

Moving on to an **Octane2** equipped with the **V10 Board**.

Because we've already built a very nice .vfo on the previous example for the **Fuel V12** system, I am going to use it with the **Octane2** as is, without any modifications.

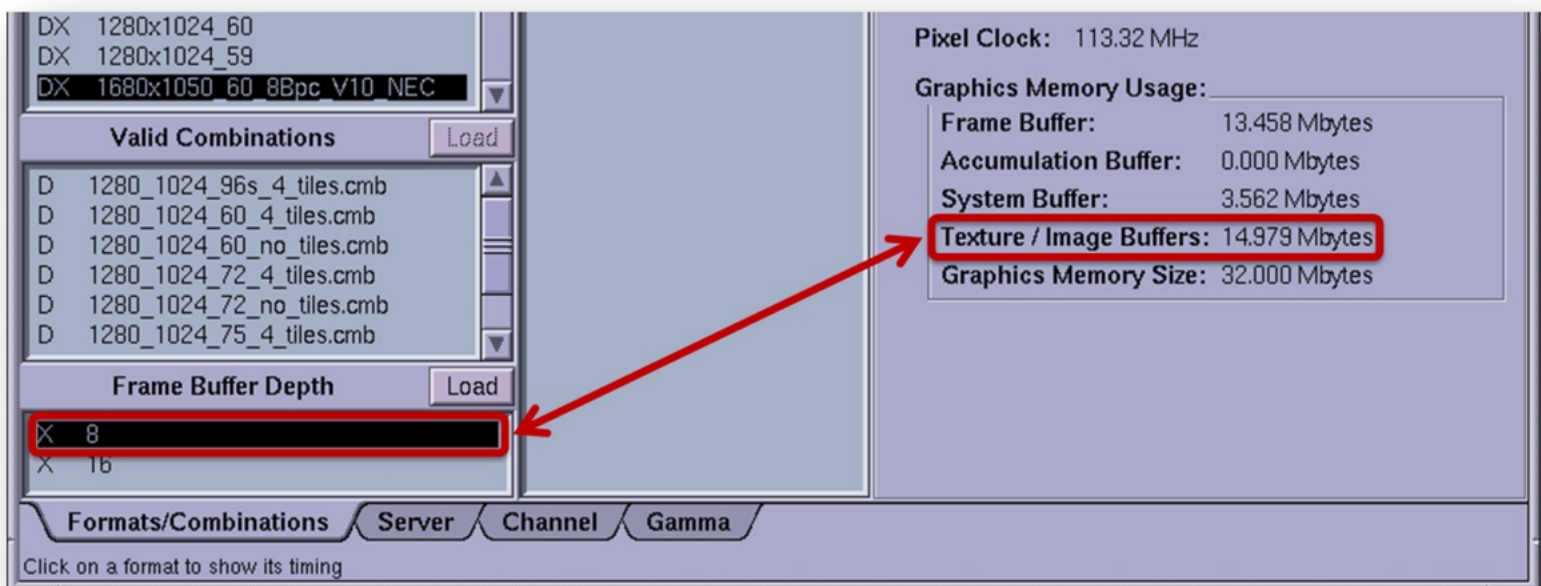
After all, it does not even need to be recompiled cause this .vfo should work with all **Odyssey Boards**.

However, in case we leave the **Frame Buffer Depth** setting at **x16** there is really insufficient space for the **Texture / Image Buffers** (as explained on the **Frame Buffer Depth** section):



Already mentioned that decreasing the **Frame Rate** or reducing the **Blanking Bezels** won't improve the size of these buffers at all.

The only way around this, is to actually decrease the (**Active**) **Resolution** of the format or lower the **Frame Buffer Depth** value:



Note: For the **V10** & **V12**, one does not have to achieve the lowest possible **Blanking** values, but on the next example, it will be made clear why sometimes we should.

Octane, V8, 1680x1050:

The first generation of **VPro Boards** (V6, V8) had a design flaw, prohibiting a bunch of commonly used resolution formats that are totally usable in other **SGI Video Boards**.

Specifically, modes whose **Pixel Clock** ranges between 109MHz and 193MHz are not considered “apply-able” and remain **hidden** from the **Valid Formats**’ list of the **Display Properties** panel.

And even then, the ones that are close enough to the bottom 109MHz of that range will only work at **Frame Buffer Depth** value **x8**, cause should we try to load them at **x16**, there will be lots of **noise** and **flickering** when moving the windows around (or even while doing nothing at all).

These **artifacts** will gradually fade away as the **Pixel Clock Frequency** gets more reduced (below 109MHz).

In my case, they almost stopped happening at ~97MHz.

Again, this is an issue relevant only to the highest **Frame Buffer Depth** setting.

In case we use the lower **x8** value, we can go as near the 109MHz bottom boundary as we want without this flickering issue.

Good thing though, the common **HD** formats: 1920x1200@60Hz and 1920x1080@72Hz fall out of the upper bound of the forbidden range.

Obviously, on a **V6**, one has to account for the **Texture / Image Buffers** size, as explained previously. So achieving these high resolutions implies having to use the **x8 Frame Buffer Depth** value.

However, the 1680x1050 format is a bit tricky.

As can be seen at the bottom of page 16, during our initial attempt, the **Pixel Clock** was running at 141.92MHz, which falls just in the middle of the forbidden zone.

There are two things that can be done to alter the **Pixel Clock** value:

- Adjust the **Frame Rate**, and/or

- Tinker with the **TotalPixelsPerLine** and **TotalLinesPerFrame** values, while keeping the **ActivePixelsPerLine** and **ActiveLinesPerFrame** at 1680 and 1050.

We could, theoretically, increase the **Frame Rate** and inflate the **Blanking Bezels** *a lot* to help make the **Pixel Clock Frequency** > 193MHz, but this would lead to a very noisy video output, plus the display would think it is being fed with a 1280x1024 signal and show a compressed, distorted image.

So! A starting point is to try to minimize the **Blanking Bezels**.

The entire relevant procedure has been shown on pages 17 through 19.

And the resulting **Pixel Clock** value would be 113.32MHz, at that point (tiniest **Blanking** possible - for the monitor I am using, at least).

To drop this frequency even more, we have to slightly lower the **Frame Rate**.

Setting it to 58Hz resulted to 108.94MHz and at **FBD x8**, it worked just fine.

On a **V6**, we would stop here, but since I am planning to use this **V8 Octane** for some **3D Modeling**, it needs to be **x16**.

As already mentioned, since we are currently very close to the 109MHz bound, applying the **x16** value will result to incredibly dense artifacting, to the point that the computer will be unusable.

Figured that the lowest possible **Frame Rate** my monitor would sync OK (without recognizing the input signal as **1280x1024** and showing a narrowed picture) happens to be **51.65Hz**.

This leads to a **97.02MHz Pixel Clock Frequency** and the disturbing artifacts are almost entirely gone. They sometimes appear while panning through **Maya's viewport**, but I'm sure that could my monitor sync at **50Hz**, I would be golden.

And there is the code I used:

```
General
{
    FieldsPerFrame = 1;
    FramesPerSecond = 51.6514;
    TotalLinesPerFrame = 1077;
    TotalPixelsPerLine = 1752;
    ActiveLinesPerFrame = 1050;
    ActivePixelsPerLine = 1680;
    FormatName = "1680x1050_52";
}

Active Line {
    HorizontalFrontPorch = 8 pixels;
    HorizontalSync = 24 pixels;
    HorizontalBackPorch = 40 pixels;
}

Field
{
    Vertical Sync =
    {
        {
            Length = 1.0H;
            Low = 0.0 usec;
        }
        repeat 23
        {
            Length = 1.0H;
        }
    }

    Initial Low;
    Vertical Back Porch =
    {
        {
            Length = 1.0H;
            High = HorizontalSync;
        }
        repeat 1
        {
            Length = 1.0H;
            Low = 0.0 usec;
            High = HorizontalSync;
        }
    }

    Initial High;

    Active =
```

The only real difference, compared to the code on pages 18-19, is the "FramesPerSecond" value.

Also, made the Vertical Sync value: 23 and the Vertical Back Porch value: 1

On the previous code, these values were:
Vertical Sync:3 and
Vertical Back Porch: 21

The TotalLinesPerFrame remain the same (1078), and so the Blanking bezels have not been modified, but with this swap I noticed slightly less flickering and streaking.

```

    {
        repeat 1050
        {
            Length = 1.0H;
            Low = 0.0 usec;
            High = HorizontalSync;
        }
    }

    Vertical Front Porch =
    {
        repeat 1
        {
            Length = 1.0H;
            Low = 0.0 usec;
            High = HorizontalSync;
        }
    }
}

#if 0
    postprocess
    {
        dump edge;
    }
#endif

```

Compiled on a console:

```
/usr/sbin/vfc -c chip=VPro_Chip.def,board=VPro_Board.def -o 1680x1050_52_x16_NEC.vfo
/usr/gfx/ucode/vfc/vfs/1680x1050_52_x16.vfs
```

Moved the **1680x1050_52_x16_NEC.vfo** to **/usr/ucode/ODSY/vof** and loaded the format via the Display Properties panel.

All relevant files can be found inside the “**Examples**” folder, included with this guide.

Onyx2, InfiniteReality3, 1680x1050:

The **Onyx2 IR Display Generator** is not subject to the limitations of other **SGI’s Video Boards**. It also does not like the extremely reduced **Blanking Bezels** we achieved so far.

As a result, we will go ahead and use the **BlockSync** template to generate a valid .vfo for the **1680x1050@60Hz** resolution format:

```
/usr/sbin/vfc -p "-DACTIVE_LINES=1050" -p "-DACTIVE_PIXELS=1680" -p "-DPFS=60" -c
chip=voc1.def,board=dg4.def -o 1680x1050_IR3.vfo /usr/gfx/ucode/vfc/vfs/BlockSync.vfs
```

(Use the above command to compile a .vfo for any **Resolution** and **Refresh Rate** you wish.)

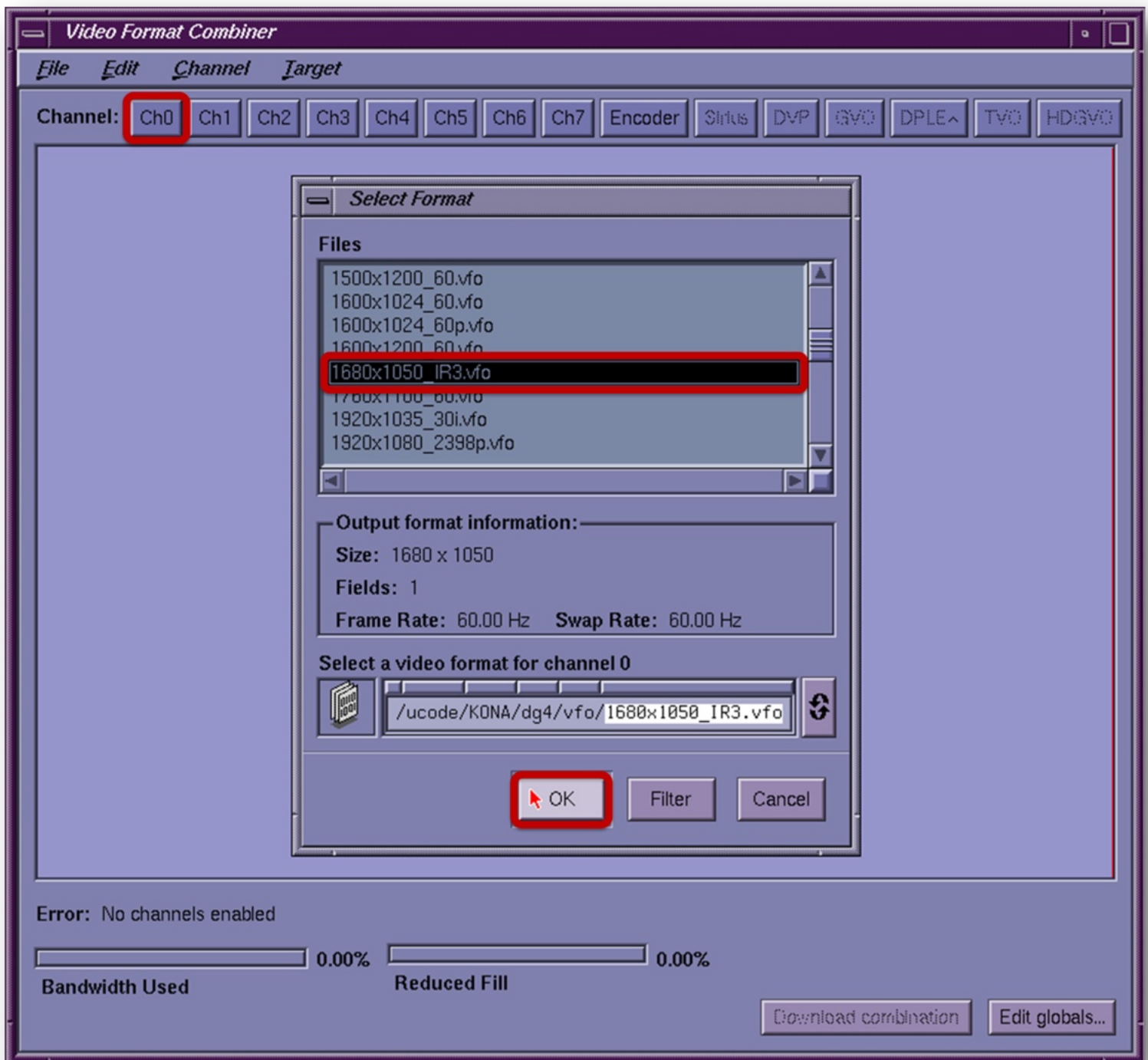
Then, we need to move the **1680x1050_IR3.vfo** from the **root (/)** folder to:

```
/usr/gfx/ucode/KONA/dg4/vfo
```

However, in order to apply it, we have to use the **ircombine** tool, located inside the **/usr/gfx** folder.

This will launch the **Video Format Compiler** window.

Click on the **Ch0** button, and on the pop-up window, select the newly created **1680x1050_IR3.vfo** and hit “**OK**”:



This tool helps you create “Combinations” with multiple monitors and different output formats on each, but this guide is only helpful for **single monitor setups**.

Moving on, select **File** → **Save To EEPROM...**

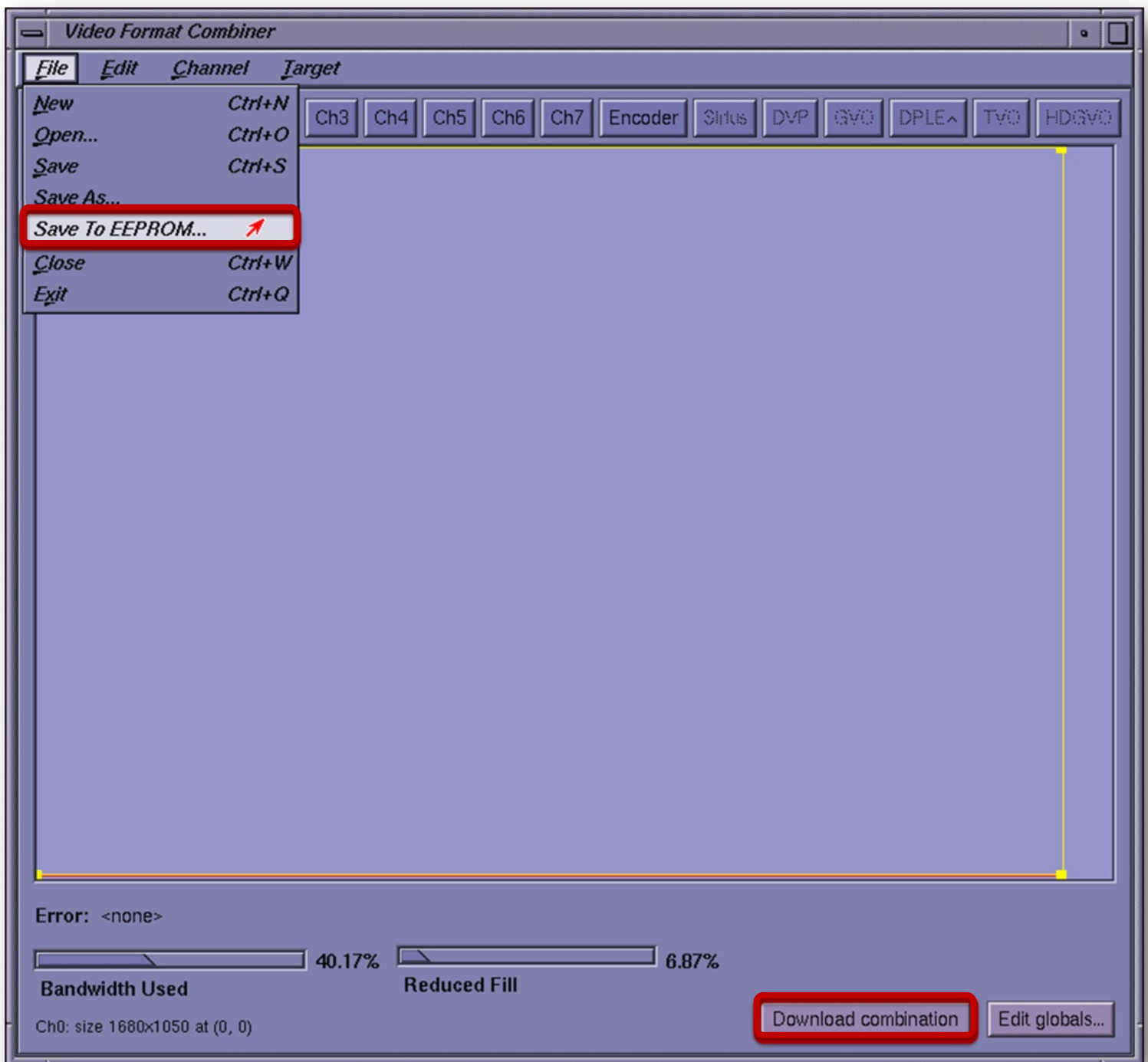
The **combination** is now **hardcoded** to the **DG** and will be loaded every time the system powers on.

Then click on “**Download Combination**” on the current window to try and apply it immediately.

In case it won’t let you load it instantly, reboot to fix!

This will happen if the current resolution has less **Vertical Active Lines** than the new one.

ie. If the running resolution is **1280x1024** and we are applying **1680x1050** it warns us to reboot cause **1050 > 1024**.



And that should be it!

Troubleshooting:

Another reason I suggest using the **BlockSync** template is that, unlike all other **SGIs** we examined, the **Onyx2** keeps and outputs this format we saved to the **EEPROM** immediately when powered up. Playing with the **blanking** or the **timing** values of a custom .vfs might lead to building a **wrong combination** that is **out of our monitor's range**.

The other systems we experimented on, power on at **1280x1024** and load the "Default" **Resolution Format** we have set, after Irix is done booting.

Onyx2 applies it immediately when switched on, giving no room for error.

That is why the troubleshooting method described in page 11 won't be saving the day.

In case you are unable to wake up your display after applying a **custom Combination**, do the following steps:

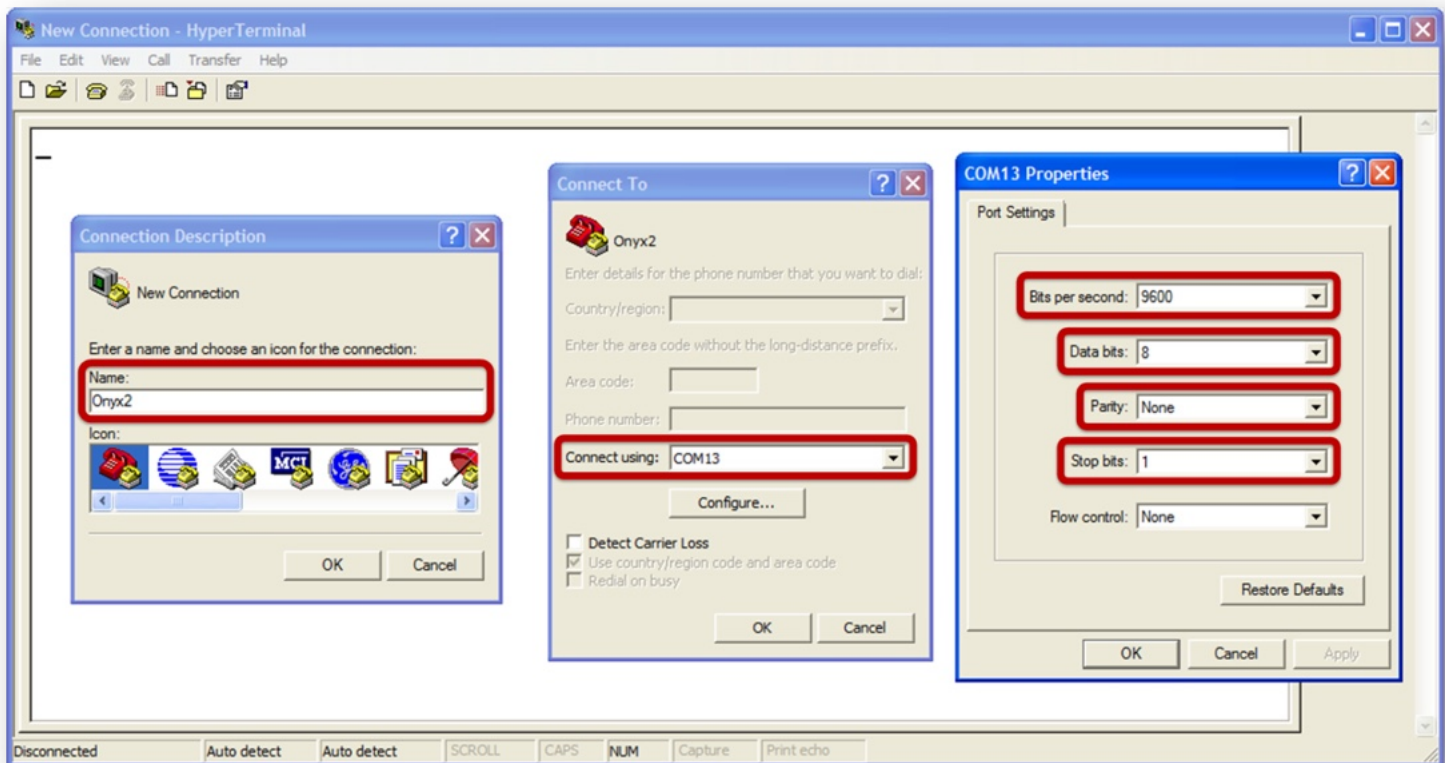
Shut down the **Onyx2**

Disconnect keyboard and mouse

Connect a 9-pin "Null Modem Cable" to the "tty_1" port on the back of the system and plug the other end to a Windows PC's serial port

Use "HyperTerminal" (more detailed instructions can be found on [Nekochan](#) or [Google](#))

Create a new **Connection** with these settings:



Name the custom connection,

Select the serial port in which the cable was connected to (usually **COM1**, but in my case **COM13**),

Set the **Bitrate** to **9600**, the **Data bits** to **8**, **no Parity**, **1 Stop Bit**, and

Keep the connection running

While the connection is up, fire up the **Onyx2** and you should be seeing its output on the **HyperTerminal**'s window on your PC

You can boot up the **Onyx** via command line through **HyperTerminal**

Wait for **Irix** to load and **login as root**

Enter: `"/usr/gfx/stopgfx"`

Enter: `"cd usr/gfx/KONA/bin"`

Then run: `"/ireeprom -c"` to **erase** and **reset** the **EEPROM** to the **failsafe 1280x1024** format

Enter: `"/usr/gfx/startgfx"`

Shut down the **Onyx2**

Reconnect the keyboard and the mouse

Quit HyperTerminal and disconnect the Null Modem Cable

Fire the Onyx up again, and it should now output a resolution of 1280x1024@60Hz to the display

But, just saying, in case we use the BlockSync template, the chance of ending up with an unusable “Default Combination” is minimal.

02, 1600x1024:

In this final example, I’m going to walk you through the process of pairing the O2 with a really old monitor coming from the ~2000 era.

It is an SGI F220, or a rebranded LG 295LM, if you prefer.

The panel’s Native Resolution is 1600x1024@60Hz.

However in the Specifications Manual, it would note that the max Resolution through DVI is 1280x1024@60Hz and through VGA would be 1600x1200@75Hz:

3. SIGNAL (Refer to the Timing Chart)

3-1. Video Input

- 1) Signal Input : 15 pin D-Sub Connector/
DVI-D Connector
- 2) Input From : Separate, RGB Analog, 0.7Vp-p/
75Ω, Positive Digital
- 3) Resolution (max.): Analog - 1600x1200@75Hz
Digital - 1280x1024@60Hz

Strange, cause these resolutions are of 4:3 aspect ratio, although the panel is a 16:10 one.

Then, on the Timing Chart, I found this Modeline for the 1600x1024@60Hz format:

Mode	H/V Sort	Sync Polarity	DOT Clock	Frequency	Total Period (E)	Video Active Time (A)	Front Porch (C)	Sync Duration (D)	Back Porch (F)	Resolution
17	H	—	112.27	62.11	1808	1600	90	30	88	1600x1024 60Hz
	V	—	(Analog Only)	60.00	1040	1024	10	3	3	

Practically means:

TotalLinesPerFrame = 1040;
TotalPixelsPerLine = 1808;
ActiveLinesPerFrame = 1024;
ActivePixelsPerLine = 1600;
HorizontalFrontPorch = 90 pixels;
HorizontalSync = 30 pixels;
HorizontalBackPorch = 88 pixels;
Vertical Sync = 3
Vertical Back Porch = 3 - 2 = 1
Vertical Front Porch = 10

Unfortunately though, it did not work after compiling.

The F220 thought it was fed with a 1280x1024 signal and would display an inwards-stretched image.

I connected this monitor to my Windows Workstation and it could not detect its Native Resolution.

Had to go to nVidia's Control Panel and force it to send a 1600x1024 signal from there.

This worked, but got me thinking that the DDC (EDID) communication between the F220 and the computer was broken.

A possible reason is that the display is really old (manufactured in 2000) and did not support EDID.

This was also the case when connected it to a Mac and tried to dump the EDID via SwitchResX:

It could not detect the Native Resolution (OSX would think it is 800x600 monitor) and there was no EDID info available.

The cables I used were good, so it has to be a thing with this particular display model.

Note that should the F220 be a "regular" modern monitor, one would have to follow the exact same process described on the previous examples..

Back to the Calculator from the <https://arachnoid.com/modelines/index.html> website, then.

The input:

Required inputs:

Horizontal resolution pixels	1600
Vertical resolution pixels	1024
Refresh rate Hz	60

The resulting Modeline:

Results:

compute

```
# 1600x1024 @ 60.00 Hz (GTF) hsync: 63.60 kHz; pclk: 136.36 MHz
Modeline "1600x1024_60.00" 136.36 1600 1704 1872 2144 1024 1025 1028 1060 -HSync +Vsync
```

In other words:

```
TotalLinesPerFrame = 1060;
TotalPixelsPerLine = 2144;
ActiveLinesPerFrame = 1024;
ActivePixelsPerLine = 1600;
HorizontalFrontPorch = 104 pixels;
HorizontalSync = 168 pixels;
HorizontalBackPorch = 272 pixels;
Vertical Sync = 3
Vertical Back Porch = 32 - 2 = 30
Vertical Front Porch = 1
```

Tried with these values but, alas, no luck either:

The monitor would detect a **1280x1024** input, plus the image seemed to be cropped.

After several attempts I came to the conclusion that the on-board “**Auto-Adjust**” function of this display is a bit primitive and fails to always center the image (regardless the resolution).

This is a thing to expect with older flat monitors.

Noticed, though, that after modifying the **Horizontal Back Porch** & the **Horizontal Sync** by **increments of 24**, I would get a full **1600x1024** signal.

However, even then, a small region of rows would be cropped and the **F220**’s controller was still unable to perfectly center and proportionally stretch the image.

The cropping issue started to disappear while the **Horizontal Back Porch** was being increased (again - by **increments of 24**).

So, reached up to this point:

```
TotalPixelsPerLine = 2120;  
HorizontalFrontPorch = 88 pixels;  
HorizontalBackPorch = 264 pixels;  
HorizontalSync = 168 pixels;
```

I suspect that should I be able to further increase the **Horizontal Back Porch**, it could result to a perfect, centered **1600x1024** image.

The **TotalPixelsPerLine** limit, however, for the **O2** is **2160**.

And things really start to break down even when this value surpasses the **2100** mark.

Heavy streaking and other artifacts start to appear on the left side of the screen and get more intense as we approach the **2160** limit.

The safest thing to do is stay below **2088**.

Finally, after a lot of experimenting, settled on these values:

```
TotalLinesPerFrame = 1070;  
TotalPixelsPerLine = 2072;  
ActiveLinesPerFrame = 1024;  
ActivePixelsPerLine = 1600;  
HorizontalFrontPorch = 88 pixels;  
HorizontalSync = 216 pixels;  
HorizontalBackPorch = 168 pixels;  
Vertical Sync = 3  
Vertical Back Porch = 42 - 2 = 40  
Vertical Front Porch = 1
```

Yeap, the **Vertical Back Porch** had to also be increased for the **F220** to be able to detect the full **1600x1024 Active Resolution** (instead of **1280x1024**).

Tip: Most of the times, increasing the **Vertical Back Porch** will contribute to help wake the monitor from thinking it is being signaled a **1280x1024** mode.

Built the .vfo:

```
/usr/sbin/vfc -c chip=O2_Chip.def,board=O2_Board.def -o 1600x1024_F220.vfo  
/usr/gfx/ucode/vfc/vfs/1600x1024.vfs
```

And the content of this particular .vfs is:

```
/* SGI F220 1600x1024@60Hz */  
  
General  
{  
    FieldsPerFrame = 1;  
    FramesPerSecond = 60;  
    TotalLinesPerFrame = 1070;  
    TotalPixelsPerLine = 2072;  
    ActiveLinesPerFrame = 1024;  
    ActivePixelsPerLine = 1600;  
    FormatName = "1600x1024_60";  
}  
  
Active Line {  
    HorizontalFrontPorch = 88 pixels;  
    HorizontalSync = 216 pixels;  
    HorizontalBackPorch = 168 pixels;  
}  
  
Field  
{  
    Vertical Sync =  
    {  
        {  
            Length = 1.0H;  
            Low = 0.0 usec;  
        }  
        repeat 3  
        {  
            Length = 1.0H;  
        }  
    }  
  
    Initial Low;  
    Vertical Back Porch =  
    {  
        {  
            Length = 1.0H;  
            High = HorizontalSync;  
        }  
        repeat 40  
        {  
            Length = 1.0H;  
            Low = 0.0 usec;  
            High = HorizontalSync;  
        }  
    }  
  
    Initial High;  
  
    Active =  
    {  
        repeat 1024
```

```

        {
            Length = 1.0H;
            Low = 0.0 usec;
            High = HorizontalSync;
        }
    }

    Vertical Front Porch =
    {
        repeat 1
        {
            Length = 1.0H;
            Low = 0.0 usec;
            High = HorizontalSync;
        }
    }

    #if 0
        postprocess
        {
            dump edge;
        }
    #endif

```

These values led to an almost perfect result of a **1600x1024@60Hz resolution** with just (I believe) **16 Vertical Rows** (maybe even less) cropped off the right portion of the screen.

Meaning: The Desktop extends about **16 Rows** past the right end of the **viewable area** of the display.

However, should this monitor have a more advanced “**Auto-Adjust**” function, this issue would have been solved.

This example was included just to show that building custom resolutions for older flat monitors can be quite trickier and more time consuming.

Conclusion:

After browsing through the **theory** and the **examples** of this guide, you should be able to **write** and **compile** your own **custom resolution**, tailor-made for your monitor of choice.

Not all possible configs have been examined, but the ones excluded (**MGras Octanes**, **Indigo2s**, etc..) can be basically tackled the same way.

Only obvious differences are the **VFC command’s syntax** & any **Video Board** hardware limitations.

Thanks for your time!